

Deploy Guide

GitHub Actions + Vultr (Streamlined)

The Big Picture

Push to production → GitHub Actions Starts → SSHs into Vultr → Pulls Code & Restarts App

The Two Directions Needed

- **GitHub → Server:** Actions needs to log in to run deployment commands. *(Uses Repository Secrets)*
- **Server → GitHub:** Server needs authorization to pull your code repo. *(Uses Deploy Keys)*

Step 1 — Server Setup (One-Time)

Log into your Vultr server and ensure the application environment is initialized:

```
# Clone and switch to production
git clone git@github.com:YOUR_ORG/YOUR_REPO.git ~/ashgard_todo
cd ~/ashgard_todo && git checkout production

# Set up Python environment
python3 -m venv venv
source venv/bin/activate
pip install -r server/requirements.txt

# Create local environment config
nano server/.env
```

Systemd Service: Make sure your `ashgard_todo` systemd service file is pre-configured and managed by your server administrator.

Step 2 — Connect Server to GitHub (for Git Pull)

1. **On the Server:** Generate a unique key pair and view the public signature.

```
ssh-keygen -t ed25519 -f ~/.ssh/github_deploy -N ""
cat ~/.ssh/github_deploy.pub
```

2. **On GitHub:** Go to **Settings → Deploy keys → Add deploy key**. Paste the public string (leave Write Access disabled).

3. **On the Server:** Route automated GitHub requests explicitly through this key.

```
nano ~/.ssh/config
```

Append the following configuration block:

```
Host github.com
  HostName github.com
  User git
  IdentityFile ~/.ssh/github_deploy
  IdentitiesOnly yes
```

Test authorization:

```
chmod 600 ~/.ssh/config ~/.ssh/github_deploy
ssh -T git@github.com
```

Step 3 — Connect GitHub to Server (for Actions SSH)

1. **On your local PC:** Create a separate key pair for GitHub Actions workflow access.

```
ssh-keygen -t ed25519 -f github_actions_key -N ""
```

2. **On the Server:** Authorize the public key signature (`github_actions_key.pub`).

```
nano ~/.ssh/authorized_keys # Paste the key string onto a new line
chmod 600 ~/.ssh/authorized_keys
```

3. **On GitHub:** Navigate to **Settings** → **Secrets and variables** → **Actions**. Create 3 Repository Secrets:

Secret Name	Value / Description
<code>SERVER_HOST</code>	Your target Vultr Server IP address (e.g., <code>123.45.67.89</code>)
<code>SERVER_USER</code>	Your dedicated SSH deployment user (e.g., <code>deploy</code>)
<code>SSH_PRIVATE_KEY</code>	The exact, full text contents of your local <code>github_actions_key</code>

Step 4 — Allow Passwordless Restart

Because GitHub Actions executes `sudo systemctl restart` , you must bypass the password prompt securely for that specific command.

On the Server:

```
sudo visudo -f /etc/sudoers.d/ashgard-deploy
```

Add this line (replace `YOUR_USER` with your actual deployment username):

```
YOUR_USER ALL=(ALL) NOPASSWD: /bin/systemctl restart ashgard_todo
```

Step 5 — Trigger the Deployment

1. Verify that `.github/workflows/deploy.yaml` is committed on your `production` branch.
2. Push or merge code directly into the `production` branch.
3. Monitor progress under GitHub's **Actions** dashboard tab.

Quick Troubleshooting

Error / Symptom	Resolution Step
<code>Permission denied (publickey)</code>	Verify <code>SSH_PRIVATE_KEY</code> secret matches your key exactly, and that the public equivalent is in the server's <code>authorized_keys</code> .
<code>git pull</code> fails on remote server	Verify the token in GitHub Deploy keys matches the local server's <code>github_deploy.pub</code> .
<code>sudo: password required</code>	Re-verify Step 4 syntax. Confirm filename and match the targeted username exactly.

Inspect real-time service logs:

```
sudo journalctl -u ashgard_todo -n 50
```

Important Reminders: Front-end asset compilation (`npm run build`) and database structural upgrades (`flask db upgrade`) are not automated in this workflow. Perform them manually or append them as needed to your workflow pipeline configuration.